4.1 Architectural Design

The architecture design process begins with a high-level view of system features, focusing on how different user groups interact with the system. This perspective then transitions into the technical architecture, detailing how these features are implemented through suitable containers, technologies, and communication flows. The system supports multiple stakeholders, each with distinct capabilities:

- **Unauthenticated Customers**: Browse the product catalogue with limited access.
- **Authenticated Customers**: Register accounts, browse products, operate shopping carts, view orders, and write reviews.
- **Vendors**: Log in with administrator accounts to manage products and orders.

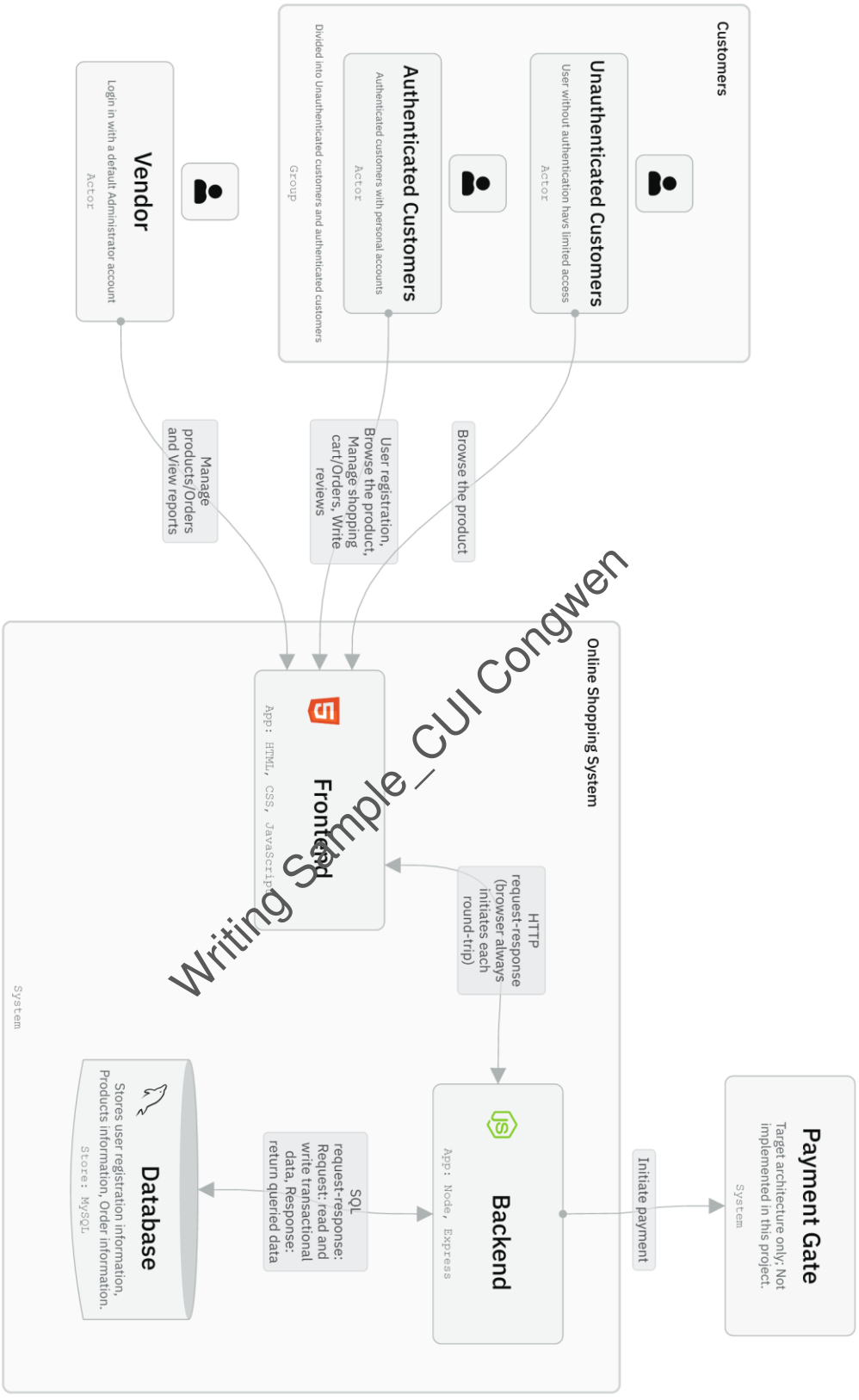
4.1.1. System Architecture

To support these features, the system is divided into three main containers: Frontend, Backend, and Database, with a planned integration to a Payment Gateway.

- **Frontend (HTML, CSS, JavaScript)**
  - Provides the user interface for customers and vendors.
  - Handles browsing, registration, order management, and review submission.
  - Initiates all interactions via HTTP requests to the backend.
- **Backend (Node.js, Express)**
  - Serves as the application server, processing requests from the frontend.
  - Implements business logic such as product management, order processing, and user authentication.
  - Communicates with the database using SQL queries to perform CRUD (Create, Read, Update, Delete) operations.
  - Initiates payment requests to the payment gateway
- **Database (MySQL)**
  - Stores user registration data, product information, and order records.
  - Responds to backend queries with transactional data, typically returned in JSON format.
- **Payment Gateway (Target Architecture)**
  - Not implemented in the current project but included for future scalability.

*Figure 5 Online shopping system architectur*e highlights the features for 4.1.1. and these containers, their responsibilities, and the communication flow among them for 4.1.2.

Writing Sample _ CUI Congwen



Writing Sample_CUI Congwen

Figure 1 Online shopping system architecture

4.1.2. Implementation Rationale

a. Front-end: Native JavaScript (ES6+), HTML5, CSS3

The frontend is built as a Single Page Application (SPA) using native JavaScript without frameworks. We chose this approach because:

- Project scope: With four team members and limited functionality, native JavaScript keeps complexity manageable. It avoids the overhead of learning and maintaining a framework, letting the team focus on core features.
- Control: Direct DOM manipulation provides full flexibility and avoids framework constraints, allowing tailored implementations for project needs.

b. Back-end: Node.js, express.js

We chose JavaScript for the backend because:

- Unified language: Same language for frontend and backend, reducing context switching and enabling code sharing.
- Performance: Non-blocking I/O and the event loop handle concurrent requests efficiently, ideal for I/O-heavy tasks.

We chose Express.js because:

- Maturity: The most popular Node.js framework, stable, well-tested, and widely supported.
- Flexibility: Unopinionated design allows easy customization and integration with middleware.
- Lightweight: Minimal overhead, fast performance, and simple maintenance.

Writing Sample _ CUI Congwen

c. Database: MySQL

We chose MySQL[6] because:

- Features: Supports foreign key constraints for referential integrity and transactions for atomic operations.
- Cost: Open-source and free, with no licensing fees.

4.1.3. Design Alternatives and Comparison

Before finalizing the system architecture, several alternative technologies were evaluated to ensure that the chosen design aligns with the project's scope, development constraints, and long-term maintainability.

For the backend framework, we compared Node.js with Express against heavier options such as Spring Boot and Django. Although these frameworks offer strong robustness and built-in tools, they introduce higher learning overhead and slower iteration cycles for a small team. Node.js with Express was selected because it supports full-stack JavaScript development, integrates naturally with JSON-based REST APIs, and provides the flexibility required for rapid development.

For the API design, REST was chosen instead of GraphQL. While GraphQL enables flexible querying, it requires additional schema management and more complex resolver logic. Since the system's operations are primarily CRUD-based with predictable data structures, REST offers a simpler and more transparent interface that fits the modular backend design.

Regarding data storage, the team evaluated MySQL, PostgreSQL, and MongoDB. Although PostgreSQL provides advanced indexing and MongoDB offers schema flexibility, the system's core entities—products, orders, ratings, and reviews—are inherently relational and require strong consistency guarantees. MySQL was therefore selected for its mature tooling, reliable ACID compliance, and suitability for transactional e-commerce workloads.

Writing Sample_CUI Congwen

Client-side rendering was preferred over server-side rendering after comparing both approaches. SSR improves initial load time and SEO but adds unnecessary complexity for a system that does not rely on search engine indexing. CSR provides smoother interactions and supports reusable components, making it more appropriate for the project's priorities.

4.2 Data Modelling

Data modelling is a key step in designing the online shopping system, defining how information is structured, stored, and related in the database. It ensures consistent representation of customer activities, product management, and order processing.

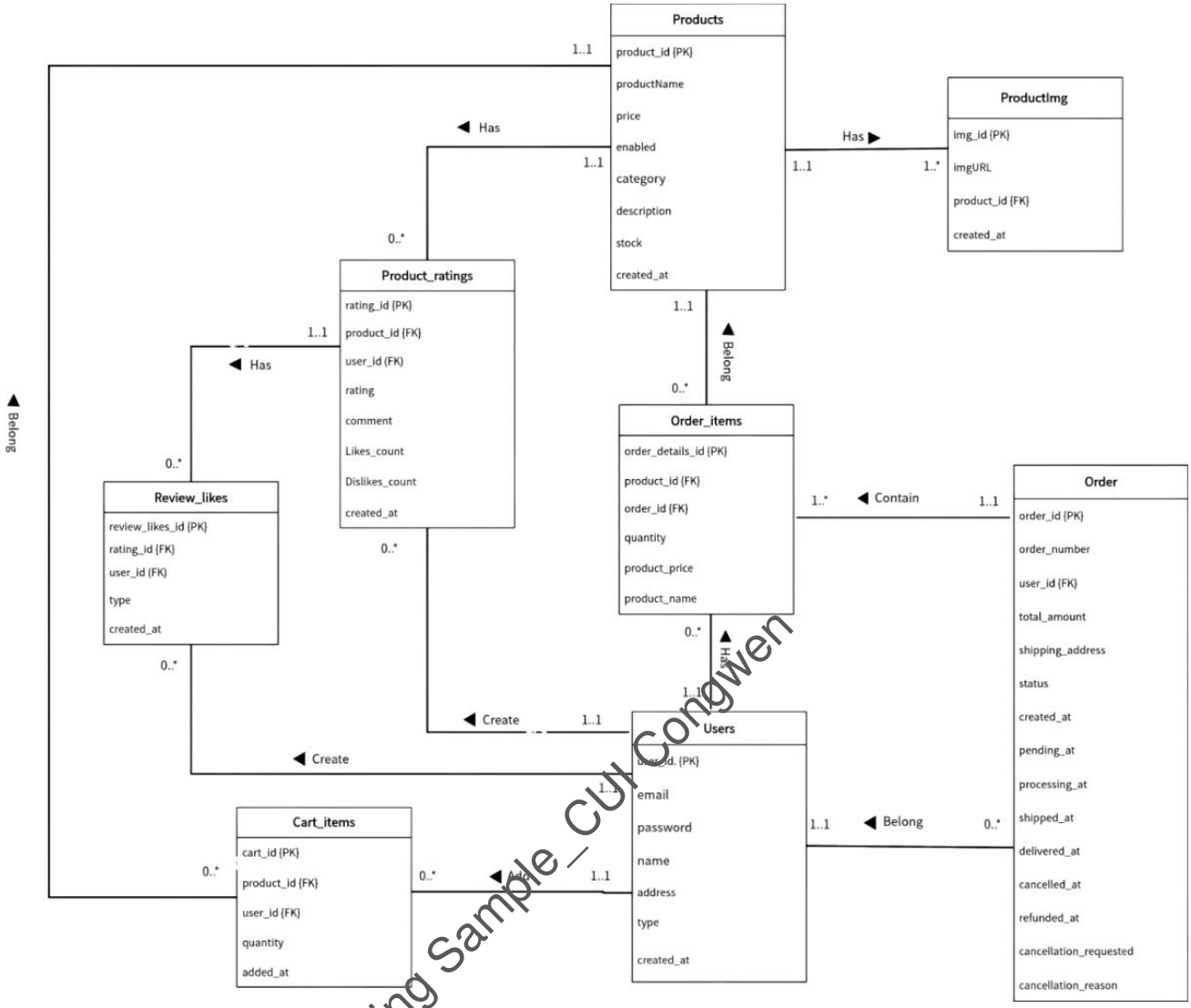
This section is divided into three parts:

- Entity–Relationship (ER) Diagram
- Table Schema Overview
- Design rationale

4.2.1. ER diagram

The diagram below shows the relationships among entities, clarifying how data flows and connects across the system.

Writing Sample_CUI Congwen



Writing Sample _ CUI Congwen

Figure 2 database schema

4.2.2. Table Schema Overview

In this part, each table's schema and design choices are explained in detail, highlighting how they support functionality and ensure data integrity.

a. User Table

Field Name	Description	Data Type	Constraint/ Default
user_id	Unique user identifier	INT	AUTO_INCREMENT
email	Unique email address used for user login	VARCHAR(100)	UNIQUE, NOT NULL
password	User password	VARCHAR(255)	NOT NULL
name	Display name of the user	VARCHAR(100)	NOT NULL
address	User address information	TEXT	

type	User role	ENUM('customer','admin')	DEFAULT 'customer'
created_at	Timestamp when the account was created	TIMESTAMP	DEFUALT CURRENT_TIMESTAMP

Table 1 User table

b. Products Table

Field Name	Description	Data Type	Constraint/ Default
product_id	Unique identifier for each product	INT	AUTO_INCREMENT
product_name	Name of the product	VARCHAR(100)	NOT NULL
price	Current unit price of the product	DECIMAL(10,2)	NOT NULL
Enabled	Indicates whether the product is available for sale	BOOLEAN	DEFAULT TRUE
category	Product category (The anime IP it belongs to)	VARCHAR(50)	
description	Detailed description of the product	TEXT	
stock	Available inventory quantity of the product	INT	DEFAULT 0
created_at	Timestamp when the product was created	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Writing Sample_CUI Congwen

Table 2 Products Table

c. Cart_items Table

Field Name	Description	Data Type	Constraint/ Default
cart_id	Unique identifier for each cart item	INT	PK, AUTO_INCREMENT
product_id	Identifier of the product added to the cart	INT	NOT NULL, FK, `products(id)` ON DELETE CASCADE
user_id	Identifier of the user who owns the cart item	INT	NOT NULL, FK, `users(id)` ON DELETE CASCADE
quantity	Number of units of the product in the cart	INT	DEFUALT 1
Added_at	Timestamp when the item was added to the cart	TIMESTAMP	DEFUALT CURRENT_TIMESTAMP

Table 3 Cart items Table

d. Order Table

Field Name	Description	Data Type	Constraint/ Default
Order_id	Unique identifier for each order	INT	PK, AUTO_INCREMENT
Order_number	Business-level unique order number used for tracking	VARCHAR(50)	UNIQUE, NOT NULL
User_id	Identifier of the user who placed the order	INT	FK, NOT NULL
Total_amount	Total monetary value of the order	DECIMAL(10,2)	NOT NULL
Shipping address	Snapshot of the shipping address at the time of ordering	TEXT	NOT NULL
status	Current status of the order (e.g., pending, processing, shipped, delivered, cancelled)	ENUM((`pending`,`processing`,`shipped`,`delivered`,`cancelled`)	DEFUALT 'pending'
Created_at	Timestamp when the order was created	TIMESTAMP	DEFUALT CURRENT_TIMESTAP
pending_at	Timestamp when the order entered pending status	DATETIME	
Processing_at	Timestamp when the order entered processing status	DATETIME	
Shipped_at	Timestamp when the order was shipped	DATETIME	
Delivered_at	Timestamp when the order was delivered	DATETIME	
Cancelled_at	Timestamp when the order was cancelled	DATETIME	
Refunded_at	Timestamp related to refund processing	DATETIME	
Cancellation_requested	Indicates whether a cancellation request has been made	TINYINT(1)	DEFUALT 0
Cancellation_reason	Description of the reason for cancellation	TEXT	

Table 4 Oder Table

Writing Sample _CUI Conswen

e. Order_items Table

Field Name	Description	Data Type	Constraint/ Default
Order_detail_id	Unique identifier for each order item	INT	PK, AUTO_INCREMENT
Product_id	Identifier of the product included in the order	INT	NOT NULL, FK
Order_id	Identifier of the order this item belongs to	INT	NOT NULL, FK
quantity	Number of units of the product in the order	INT	NOT NULL
Product_price	Snapshot of the product price at the time of purchase	DECIMAL(10,2)	NOT NULL
Product_name	Snapshot of the product name at the time of purchase	VARCHAR(100)	NOT NULL

Table 5 Order items Table

Writing Sample – CUI Congwen

f. Product_img Table

Field Name	Description	Data Type	Constraint/ Default
Img_id	Unique identifier for each product image	INT	PK, AUTO_INCREMENT
Img_URL	URL of the product image	VARCHAR(500)	NOT NULL
Product_id	Identifier of the product associated with the image	INT	NOT NULL
Created_at	Timestamp when the image was added	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 6 Product img Table

g. Product_ratings Table

Field Name	Description	Data Type	Constraint/ Default
Rating_id	Unique identifier for each record	INT	PK, AUTO_INCREMENT
Product_id	Identifier of the product associated with the user	INT	NOT NULL, PK
User_id	Identifier of the user associated with the product	INT	NOT NULL, PK
rating	Numeric rating given by the user for the product	TINYINT	NOT NULL
comment	Textual review content provided by the user	TEXT	

Likes_count	Stores the total number of "like" reactions for the product review.	INT	DEFAULT 0
Dislikes_count	Stores the total number of "dislike" reactions for the product review.	INT	DEFAULT 0
Created_at	Timestamp when the review was created	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 7 Product ratings Table

h. Review_likes Table

Field Name	Description	Data Type	Constraint/ Default
Review_likes_id	Unique identifier for each like/dislike	INT	PK, AUTO_INCREMENT
Rating_id	Reference to the target product review	INT	NOT NULL, FK, product_ratings(id), ON DELETE CASCADE
User_id	Reference to the user who reacted	INT	NOT NULL, FK, users(id), ON DELETE CASCADE
Type	Reaction type (like or dislike)	ENUM('like','dislike')	NOT NULL
Created_at	Timestamp when the reaction was recorded	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table 8 Review likes Table

Writing Sample_CUI Congwen

4.2.3. Design rationale

This section does not aim to present the complete source code or the general design logic of the system. Instead, it highlights our team's specific considerations and design decisions based on the chosen product, focusing on how common commands were integrated and adapted to fit the intended functionality.

- **TINYINT:** The `enabled` field in the **Product table (**Figure 7) uses the **TINYINT** data type because it only stores Boolean-like values (0 or 1). Unlike a standard INT (4 bytes), TINYINT requires just 1 byte, reducing storage and improving database efficiency.
- **STOCK default 0:** The stock field in the **Product table (**Figure 7) defaults to **0** to prevent products from being sold before inventory is confirmed. Vendors must update the stock quantity explicitly, ensuring data accuracy and reducing the risk of selling unavailable items. This design improves reliability in the purchasing process.

- **Storing the single product price in Order_item Table** (Figure 8): Although total_amount is stored in the ORDER table, the ORDER_ITEMS table must also record product_price. The total shows overall cost, but item-level prices clarify each product contribution, support refunds, discounts, and audits, and preserve historical accuracy. Relying only on the PRODUCTS table would be inconsistent since prices change over time. Capturing item-level prices ensures accuracy, traceability, and reliability of transactions.
- **Rating and Comment Constraints: In the Product_ratings table**(Figure 9), rating is NOT NULL because it is the core of a review, providing standardized data for averages and comparisons. The comment field is optional, as writing text requires more effort and some users prefer quick ratings. This design ensures essential evaluation data is always captured while allowing flexibility for user convenience.
- **Record the state information into the Order Table (**Figure 10)**:**
    - In the current design, each order has one row in the order table. All state fields, such as *shipped_at* or *delivered_at*, are stored as columns. This makes queries simple, but many fields may stay **NULL** if the state has not happened.
    - If the design were normalized, state updates would go into a separate table. Each update would be a new row, so the same *order_id* would appear multiple times. This reduces NULL fields but creates row duplication and more complex queries.
    - The trade-off is clear: one table avoids row duplication but has many NULLs, while a separate table reduces NULLs but adds row duplication. Because the system is small, keeping state information in the order table is preferred for simplicity.

Writing Sample - CUI Congwen

The corresponding code is shown below:

```
CREATE TABLE `products` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `price` decimal(10,2) NOT NULL,
  `description` text,
  `image_url` varchar(500) DEFAULT NULL,
  `category` varchar(50) DEFAULT NULL,
  `stock` int DEFAULT '0',
  `sales_count` int DEFAULT '0',
  `enabled` tinyint(1) DEFAULT '1',
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
```

Figure 3 Code snippet of Product Table

```
CREATE TABLE `order_items` (
  `id` int NOT NULL AUTO_INCREMENT,
  `order_id` int NOT NULL,
  `product_id` int NOT NULL,
  `product_name` varchar(100) NOT NULL,
  `product_price` decimal(10,2) NOT NULL,
  `quantity` int NOT NULL,
```

Figure 4 Code snippet of Order_item Table

```
CREATE TABLE IF NOT EXISTS product_reviews (
    id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    user_id INT NOT NULL,
    rating TINYINT NOT NULL,
    comment TEXT,
```

Writing Sample_CUI Congwen

Figure 5 Code snippet of Product_reviews Table

```
CREATE TABLE `orders` (
  `id` int NOT NULL AUTO_INCREMENT,
  `order_number` varchar(50) NOT NULL,
  `user_id` int NOT NULL,
  `customer_name` varchar(100) NOT NULL,
  `total_amount` decimal(10,2) NOT NULL,
  `shipping_address` text NOT NULL,
  `status` enum('pending','processing','shipped','delivered','cancelled') D
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  `pending_at` datetime DEFAULT NULL,
  `processing_at` datetime DEFAULT NULL,
  `shipped_at` datetime DEFAULT NULL,
  `delivered_at` datetime DEFAULT NULL,
  `cancelled_at` datetime DEFAULT NULL,
  `refunded_at` datetime DEFAULT NULL,
```

Figure 6 Code snippet of Order Table

In conclusion, this database design balances integrity, efficiency, and scalability by clearly defining entities and their relationships. It captures one-to-many connections such as between

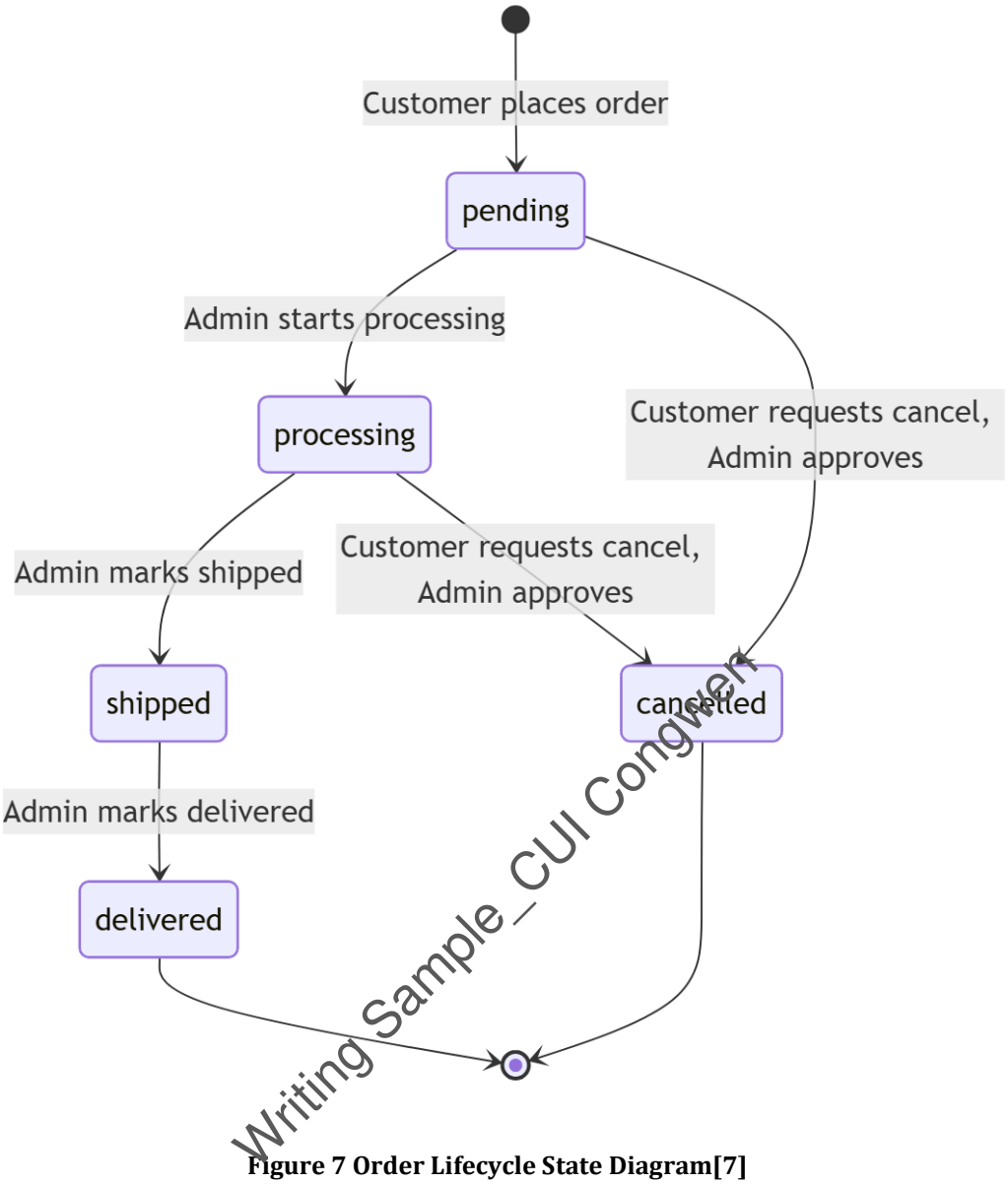
Users and Orders, Orders and Order_Items, and Products and Product_Images, and it also models many-to-many relationships between Users and Products through Cart_Items and Product_Ratings. Together, these structures ensure accurate transaction tracking, support evolving product information, and reflect diverse user behaviors. The design remains practical, reliable, and extensible, preserving historical accuracy while accommodating future growth.

4.3 Dynamic Modelling

Dynamic modelling illustrates how the system behaves during runtime, focusing on user interactions and transaction flows. Two key scenarios are captured in the diagrams:

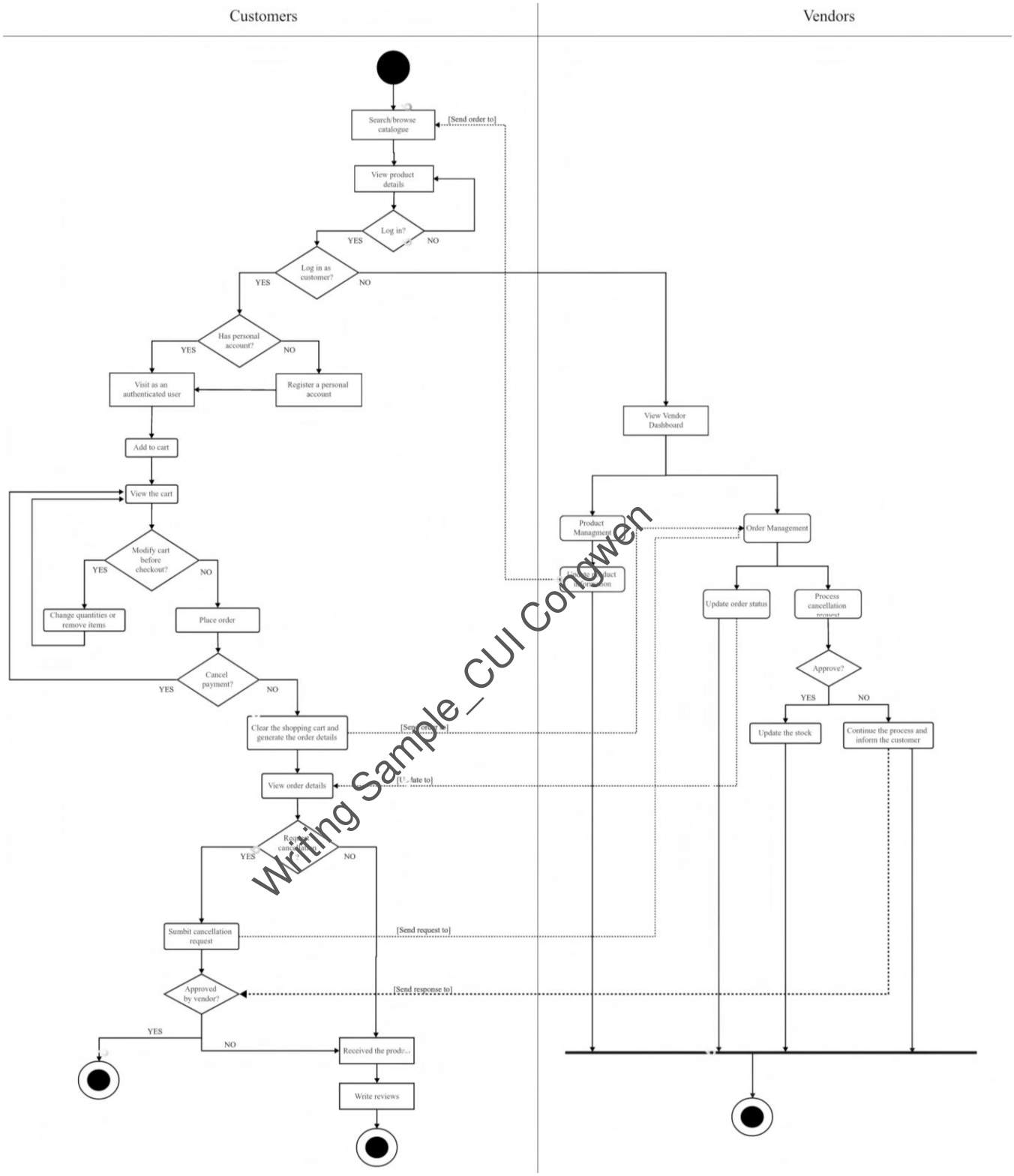
- Figure 11: **Order Lifecycle State Diagram** represents the lifecycle of an order. After a customer place an order, it enters the *pending* state. Vendors then move it to *processing*, from which it can transition to *shipped* and finally *delivered*. Alternatively, if a customer requests cancellation and the vendor approve, the order transitions to *cancelled*. These states ensure that order progress is consistently tracked in the database through the OrderStatus attribute.
- Figure 12: **Customer–Vendor Interaction Sequence diagram** shows the sequence of actions between customers and vendors. On the customer side, users can search items, view product details, log in or register, add items to the cart, place orders, request cancellations, and submit reviews. On the vendor side, administrators use the dashboard to manage products, update stock, process orders, approve cancellations, and confirm delivery. Dotted lines indicate communication flows, such as order submission or cancellation requests.

Writing Sample_CUI Congwen



Writing Sample _ CUI Congwen

Figure 7 Order Lifecycle State Diagram[7]



Writing Sample_CUI Congwen

Figure 8 Customer–Vendor Interaction Sequence

4.4 Other Design Considerations

Beyond architecture, data, and dynamic models, our design also adds two features: **user-generated content** and **accessibility**.

This section is divided into two parts—user-generated content and accessibility—showing how the system goes beyond core shopping functions to be more inclusive and responsive to customer needs

4.4.1. User-Generated Content

Since user-generated content is a functional requirement, it has already been addressed in Section 4.2 (Data Modeling) and Section 4.3 (Dynamic Modeling). In the Data Modeling section, the database design principle was explained -- specifically, that each piece of user-generated content must include a single rating, which cannot be null, while comments may remain optional to improve efficiency and user convenience. In the Dynamic Modeling section, the related process flows were described. Therefore, this part will not be repeated here.

Thus, instead of further discussing the database design, this section will focus on the implementation of the code logic.

Writing Sample _ CUI Congwen

a. Use row level locks [8] **to handle likes/dislikes and ensure concurrency safety**

For high concurrency like/dislike operations, the system adopts a consistency strategy based on **transactions, low-level locks, and state machine delta updates**.

In Review.likeReview, the back-end initiates a transaction and locks the target review row with SELECT ... FOR UPDATE (figure 13), ensuring serialized updates and preventing lost changes.

The system then checks review_likes to determine the user's current reaction state and applies one of three transitions (figure 14): first reaction (insert and increment), repeated same reaction (delete and decrement), or reaction switch (increment one counter and decrement the other).

Both **review_likes** and **product_ratings.likes_count/dislikes_count** are updated within the same transaction, with rollback on failure to maintain consistency. (figure 15)

The UNIQUE(review_id, user_id) constraint further prevents duplicate entries and insert races, ensuring reliable counts under heavy traffic and enabling immediate front-end feedback

```
static async likeReview(reviewId, userId, type) {
    const connection = await db.getConnection();

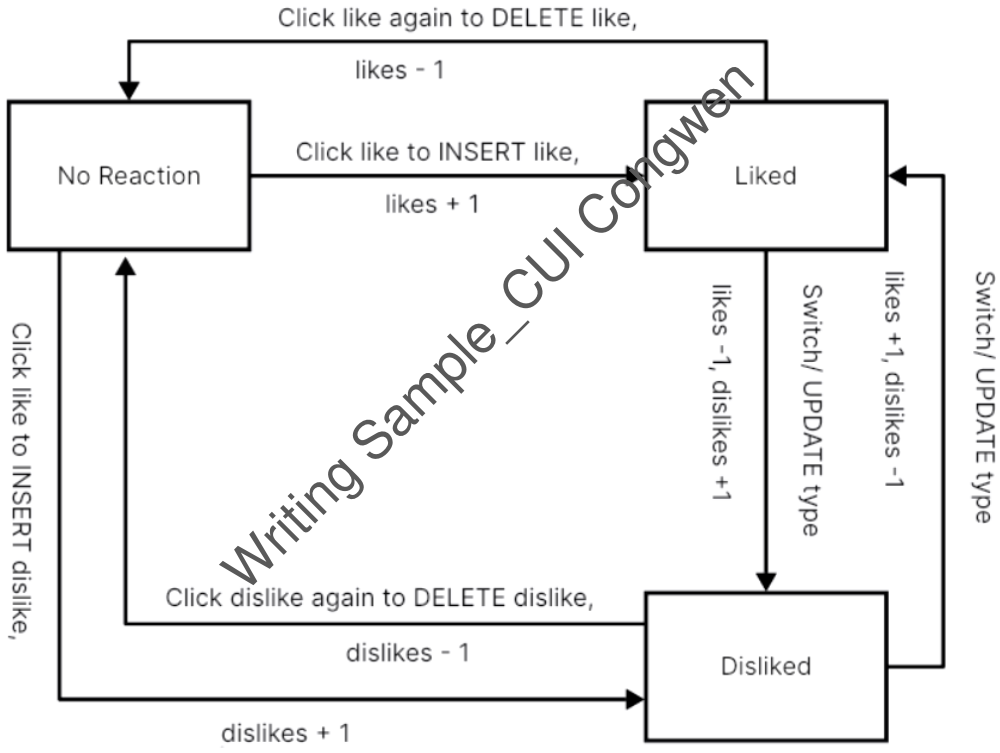
    try {
        await connection.beginTransaction();

        const [review] = await connection.query(
            'SELECT * FROM product_ratings WHERE id = ? FOR UPDATE',
            [reviewId]
        );
```

Define as asynchronous

Apply row-level rock

Figure 9 Asynchronous Transaction and Row-Level Locking in likeReview



Writing Sample_CUI Congwen

Figure 10 Per-User Review Reaction State Machine[7]

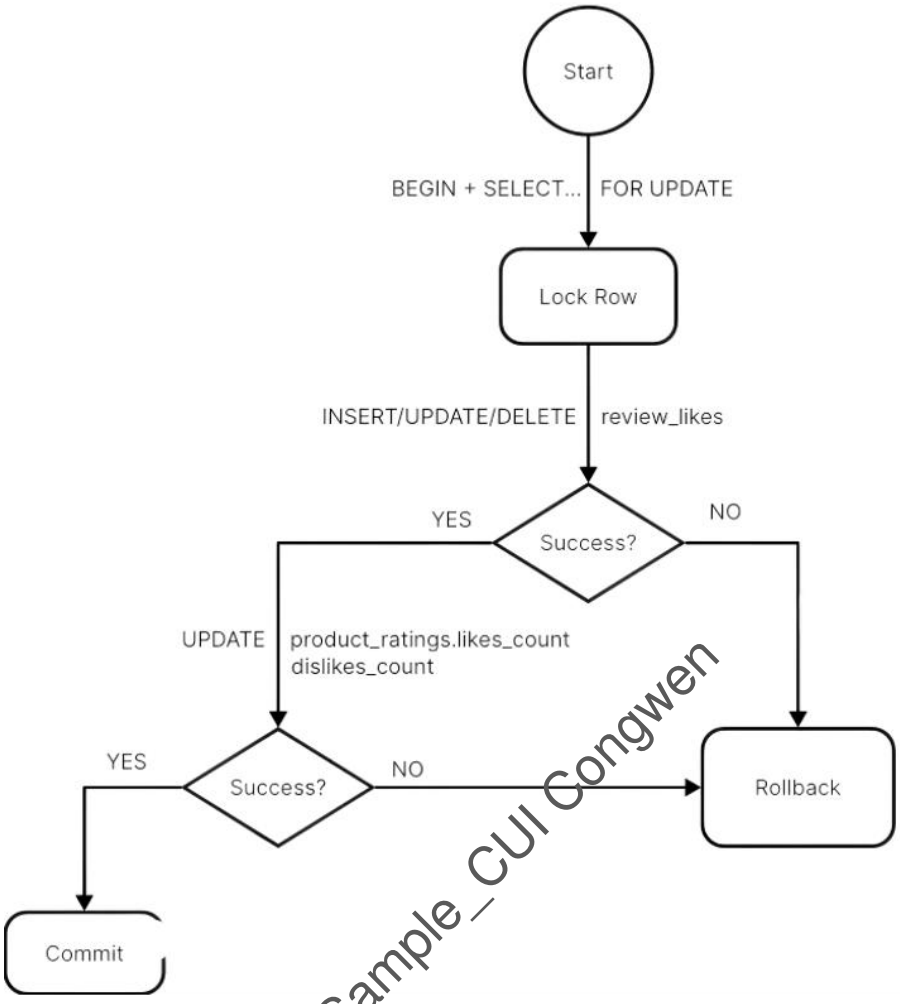


Figure 11 Transaction Flow for review_likes and Aggregated Counters

Writing Sample _ CUI Congwen

b. One-User-One-Review: Dual Guard (Data Layer + Application Layer)

The system must ensure that each customer has at most one current review row per product. That prevents spam-style duplicate reviews, keeps aggregate ratings stable, and allows users to revise their opinion (update the same row) instead of creating redundant records.

- **Data-layer hard constraint (**Figure 16). The product_ratings table declares a composite unique key on (product_id, user_id). The database therefore rejects any second insert with the same product–user pair, regardless of which client or request path issued to it.
- **Application-layer soft guard (**Figure 17). Review.createOrUpdate implements an upsert-style flow: it first checks whether a row already exists for the given product and user; if so it updates rating and comment, otherwise it inserts a new row. Under normal operation, the API behaves as “create or update my review,” which is smoother for the client than relying on database errors alone.

```
CREATE TABLE IF NOT EXISTS product_ratings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT NOT NULL,
    user_id INT NOT NULL,
    rating TINYINT NOT NULL,
    comment TEXT,
    likes_count INT DEFAULT 0,
    dislikes_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uniq_product_user (product_id, user_id),
```

Figure 12 Hard guard: UNIQUE(product_id, user_id) on product_ratings

```
static async createOrUpdate(productId, userId, rating, comment) {
    try {
        const [existing] = await db.query(
            'SELECT id FROM product_ratings WHERE product_id = ? AND user_id = ?',
            [productId, userId]
        );

        if (existing.length > 0) {
            await db.query(
                'UPDATE product_ratings SET rating = ?, comment = ? WHERE product_id = ? AND user_id = ?',
                [rating, comment, productId, userId]
            );
            return { updated: true };
        } else {
            const [result] = await db.query(
                'INSERT INTO product_ratings (product_id, user_id, rating, comment) VALUES (?, ?, ?, ?)',
                [productId, userId, rating, comment]
            );
            return { id: result.insertId, created: true };
```

Figure 13 Soft guard: Review.createOrUpdate

Writing Sample – CUI Congwen

4.4.2. Accessibility

Accessibility is an essential consideration in the design of our system, ensuring that all users, including those with diverse abilities, can interact with the platform effectively.

This section highlights several **key accessibility features worth discussing**. Common implementations such as screen reader support are not elaborated here, as they are widely adopted and well understood.

1. **Color Selection and Accessibility**

The system's color scheme follows accessibility requirements in Section 3.2, specifically WCAG 1.4.3 Contrast, which requires a minimum ratio of 4.5:1.

All text and background combinations are chosen to meet this standard by using [9] https://webaim.org/resources/contrastchecker/ . This approach guarantees readability for all users, especially those with visual impairments, while maintaining a unified visual standard across the interface.

Figures below illustrate part of the contrast ratio in our website:

- **Figure 18:** Text on main interface (Foreground #1E293B & Background #F8FAFC)
- **Figure 19:** Primary button (Foreground #F8FAFC & Background #1D4ED8)
- **Figure 20:** Error message (Foreground #F8FAFC vs Background #E11414)
- **Figure 21**: Warning message (Foreground #1E293bB vs Background #F59E0B)

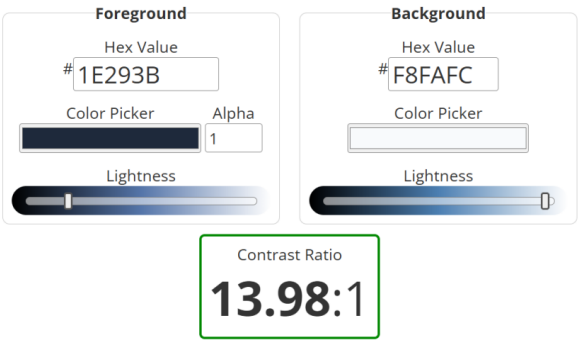


Figure 14 Primary button



Figure 15 Text on main interface



Figure 16 Warning message

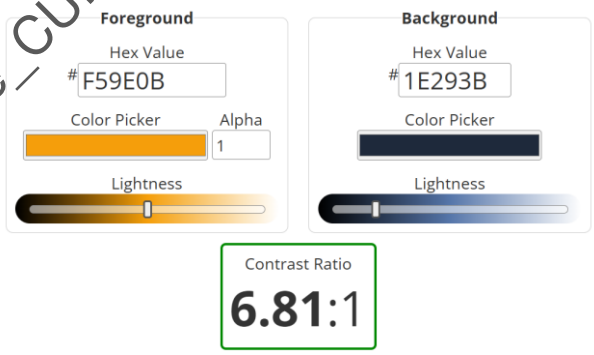


Figure 17 Error message

Writing Sample _ CUI Congwen

All the color schema has been verified through the method, which will not show thoroughly here. This approach improves maintainability and ensures that all UI components follow a unified visual standard.

2. Implementation of Multilingual Support

Given the relatively small scale of this project, a lightweight approach is adopted for multilingual support rather than complex internationalization frameworks. Our implementation has three parts:

- **Static interface elements**: Buttons, navigation, and prompts are switched between Chinese and English using a dictionary mapping. (Figure 22)
- **Product data entry**: Vendors should provide both Chinese and English names when adding products, ensuring accuracy since translation APIs may not reliably capture product names. (Figure 23)
- **User-generated content translation:** Reviews and comments are translated dynamically using the Google Translate API, providing broad coverage and accessibility (Figure 24)

A key rationale for requiring vendors to provide both Chinese and English names is that certain product names—such as anime characters—may not be translated accurately by APIs. By capturing these names directly from vendors, the system ensures correctness where automated translation may fail, thereby improving accuracy and reliability in multilingual support.

Writing Sample — CUI Congwen

```
const dict = {
  // 导航与通用
  'Home': '首页',
  'Cart': '购物车',
  'Orders': '我的订单',
  'Admin': '后台',
  'Login': '登录',
  'Logout': '退出',
  'Customer Login': '客户登录',
  'Admin Login': '管理员登录',
  'Login as Admin': '以管理员身份登录',
  "Don't have an account?": '还没有账号？',
```

Figure 18 Translation mapping



Figure 19 Vendor interface

```
async function googleTranslateGeneric(text, target = 'zh-CN') {
  const key = `${target}::${text}`;
  if (translateCache.has(key)) return translateCache.get(key);
  const url = 'https://translate.googleapis.com/translate_a/single'
    + '?client=gtx&sl=auto&tl=' + encodeURIComponent(target) + '&dt=t&q='
    + encodeURIComponent(text);
```

Figure 20 Code snapshot of invoking Google translation API

3. Rationale for Not Implementing Font Scaling

W3C [10] recommends designing pages that remain readable without relying on custom text-size controls, encouraging the use of browser-native zoom instead. In line with this principle, the system does not include built-in font scaling features, as modern browsers and operating systems already provide reliable and consistent text resizing tools through zoom functions and accessibility settings. Rather than duplicating functionality that users already have, the system focuses on accessibility areas that require deliberate design, such as color contrast and multilingual support.

Writing Sample_CUI Congwen